Directories to Data

Houston City Directory Digitization Pipeline: Final Report

Abhirami Kathirvel
Fondren Fellows Internship, Summer 2026
August 2026

1. Executive Summary

This project converts scanned pages of the 1900-1901 Houston city directory into structured, searchable data. The work is organised into six notebooks. Four of them build and validate a parsing recipe against one hand-verified page, one applies that recipe across the full 80-page directory, and one converts the result into two end-user products.

The principal results are as follows.

- 80 pages were processed end to end. Of these, 75 contained real entries, and 5 were correctly recognised as full-page advertisements or dividers containing zero entries.
- 5,621 structured directory entries were extracted, each carrying up to 14 fields: name, occupation, employer, address type, racial marker as printed, ownership and notes.
- 159 entries, or 2.8% of the total, were flagged for manual review. The flags divide into a fragment signal and a duplicate signal, and both were checked against the real data rather than assumed correct.
- Two products were delivered, both regenerated by a single notebook: a self-contained interactive HTML dashboard and a companion Excel workbook.
- 12 distinct bugs were found and fixed over the course of the project. Several of them were silently losing or corrupting real data before they were caught, and all are documented in Section 9 rather than smoothed over.

A note on scope. This report describes the pipeline as it exists at the end of the internship, including work and fixes made after the earlier check-in reports. The figures in Sections 5 and 6, covering accuracy testing and the few-shot correction attempt, reflect the runs actually made on page 200 at the time. They were not re-run for this report, and Section 11 recommends doing so against the current, fully hardened pipeline.

2. Pipeline Structure

The pipeline runs in six notebooks, in two halves. Steps 1 through 4 were performed once, against a single hand-verified page (page 200), in order to build and validate the parsing approach before it was trusted on new data. Step 5 is the production step: it applies the validated recipe to new, unseen pages, unattended, at full scale. Step 6 takes whatever Step 5 has produced and converts it into the two delivered products.

Figure 1. The six-notebook pipeline, from page to product.

#	Notebook	Purpose	Key output
1	step1_fix_boxes	Manually correct one page's auto-detected entry boundaries.	corrected_boxes.json, 79 boxes
2	step2_type_ground_truth_v3	Hand-type every field for that page's 79 entries, in the pipeline's own schema.	ground_truth.json, 79 verified entries
3	step3_accuracy_comparison	Run the pipeline on that page and score every field against ground truth.	70.3% overall field accuracy, 152 mismatches
4	step4_correction_loop	Attempt to fix the mismatches with targeted few-shot examples.	Mixed and partly negative result (Section 6)
5	step5_process_new_pages_14	Apply the hardened pipeline to all 80 pages of the source PDF.	5,621 structured entries
6	step6_build_dashboard	Compute relationships and quality flags, then build the dashboard and workbook.	Directory_Dashboard.html and .xlsx

3. Step 1: Fixing the Boxes

The existing OpenCV heuristic draws a first-pass bounding box around each directory entry on a page, but that first pass is not reliable. Boxes overlap, entries are missed entirely, and two or three entries are sometimes merged into a single box. Any text extraction built on top of bad boxes inherits all of that, so the foundation was corrected first, on a single page, before anything else was built on top of it.

What the notebook does

The notebook runs the OpenCV heuristic to obtain a starting set of boxes, then opens a purpose-built browser tool, box_editor.html, in which those boxes are corrected by hand. The tool supports three operations.

- Auto. The heuristic draws its best-guess boxes as a starting point.
- Manual edit. A box can be dragged to move it, resized by its corner or edge, created by holding Shift and dragging on empty space, or removed by selecting it and pressing Delete.
- Save. The corrected boxes are downloaded as JSON.

The saved file contains only box geometry, in full-page pixel coordinates. It holds no text and no fields, and it becomes the input to Step 2.

Output

corrected_boxes.json, holding 79 corrected boxes for page 200, together with corrected_overlay.png, which draws the corrected boxes on the source page image for a quick visual check before proceeding.

4. Step 2: Ground Truth

With the boxes for page 200 corrected, the next step was to hand-type the actual answer for every field of every entry on that page. This became the single most useful artefact in the project. It is the fixed answer key that Step 3 measures against, and the example set from which Steps 4 and 5 draw few-shot examples.

What the notebook does

A browser-based data-entry tool displays each corrected box as a cropped image beside a form for its fields, with a reference panel of sample entries kept visible at the top so that the raw-form convention stays consistent throughout typing.

One rule matters more than any other: type what is printed, not what it means. If the page reads (c), the ground truth records (c), not "colored" or "Colored". If it reads servt, the ground truth records servt, not "Servant". The pipeline expands abbreviations later using lookup tables. Ground truth has to record what is actually on the page, so that every later comparison measures the same thing on both sides.

This notebook is a revision, v3, of an earlier version. It adds four fields that were being lost under the original schema.

- residence_qualifier, for directional and positional qualifiers such as "over", "nr", "cor", "es", "ws" and "bt".
- business_name, for business or firm entries that carry no personal name.
- entry_type, taking the values person, business, institution or cross_reference.
- notes, free text for anything else, including widow status, cross-references and unusual remarks.

Output

ground_truth.json, holding 79 fully labelled entries across 14 fields, together with a crops folder containing one PNG per verified entry. A sanity-check cell reloads the saved file and prints a schema-aware summary before it is used downstream.

5. Step 3: Accuracy Comparison

With a real answer key available, this notebook runs the pipeline on page 200 using the corrected boxes and compares its output against ground truth field by field. The question is not how many entries look approximately right, but which of the 14 fields the pipeline is actually getting wrong. That distinction is what made the next two steps possible, because without a per-field breakdown there is no way to know what to target.

Results

Overall field-level accuracy across 81 pipeline-parsed entries was 70.3%, with 152 individual mismatches.

Field	Correct	Wrong	Total	Accuracy
entry_type	74	7	81	91.4%
last_name	70	9	79	88.6%
business_name	2	0	2	100.0%
first_name	61	17	78	78.2%
occupation	47	16	63	74.6%
race	27	11	38	71.1%
employer	28	14	42	66.7%
workplace_address	4	2	6	66.7%
ownership	9	6	15	60.0%
residence	30	43	73	41.1%
notes	4	9	13	30.8%
boarding	3	8	11	27.3%
residence_qualifier	1	3	4	25.0%
rooms	0	7	7	0.0%

The structural fields, meaning entry type, last name and business name, were already strong. The consistent weak spot was the group of address-type fields. This directory's notation uses three different concepts, "r." and "h." for residence, "bds" for boarding and "rms" for rooms, and the pipeline was frequently placing a value in the wrong one of the three, or dropping the qualifying detail entirely.

Output

pipeline_output.csv holding the raw results, field_comparisons.csv holding every individual comparison, mismatches.csv holding the 152 specific mismatches with ground truth against pipeline output, and per_field_accuracy.csv holding the table above.

6. Step 4: The Correction Loop

This step is worth reporting in full because it did not work as expected, and the reason it did not work is what shaped the way Step 5's prompt is actually built.

What the notebook does

The 152 mismatches from Step 3 collapse into five systematic patterns.

Pattern	Errors	Root cause
Empty entries	approx. 42	OCR missed three entries entirely, boxes 35 to 37
Residence, boarding and rooms confusion	approx. 58	Pipeline places bds and rms addresses into residence instead of the correct field
Race parentheses stripped	approx. 11	Pipeline returns c instead of (c)
Occupation and employer boundary	approx. 30	Pipeline merges employer information into occupation
Encoding artefacts	approx. 11	UTF-8 faults in the ground truth JSON itself, from the way the Step 2 tool saved the file

Between eight and ten few-shot examples targeting these patterns were written, added to the parsing prompt, and the pipeline was re-run on page 200 to measure the change.

Results: a mixed outcome

Total mismatches did fall, from 152 to 108. Macro-average accuracy across all 14 fields nevertheless became worse, dropping from 86.6% to 77.5%, because several fields that were already working regressed sharply.

Field	Before	After	Change
residence_qualifier	96.3%	50.0%	-46.3%
notes	88.9%	53.8%	-35.0%
ownership	92.6%	71.4%	-21.2%
rooms	91.4%	71.4%	-19.9%
employer	82.7%	69.0%	-13.7%
race	86.4%	73.7%	-12.7%
occupation	80.2%	76.2%	-4.1%
first_name	79.0%	78.2%	-0.8%
last_name	88.9%	88.6%	-0.3%
entry_type / business_name	91.4% / 100%	unchanged	no change
boarding	90.1%	100.0%	+9.9%
workplace_address	97.5%	100.0%	+2.5%
residence	46.9%	60.7%	+13.7%

The before-figures differ slightly from those in Section 5 because this comparison used a cleaned ground truth file, ground_truth_clean.json, produced after the encoding artefacts had been fixed, rather than the original file.

Key finding

Few-shot examples alone can overcorrect. Nudging the model toward the specific examples shown did improve the two or three fields that those examples emphasised, but it destabilised several fields having nothing to do with the patterns being targeted, some of them by double digits.

This is the direct reason that Step 5's parsing prompt, described in Section 7.4, is built on explicit and unambiguous written rules rather than relying on few-shot examples to teach the correction. Rules do not carry the same risk of overfitting to whatever a small example set happens to emphasise. Few-shot examples drawn from the Step 2 ground truth are still included in Step 5's prompt, but as supplementary context alongside the rules rather than as the mechanism doing the actual correcting.

Output

enhanced_pipeline_output.csv and enhanced_comparisons.csv holding the results under the few-shot prompt, before_after_accuracy.csv holding the table above, remaining_mismatches.csv holding the 108 mismatches still present, and few_shot_examples.json.

7. Step 5: The Production Pipeline at Scale

This is the step that runs on real, previously unseen pages, across the entire 80-page source PDF. It carries every lesson from Steps 1 to 4: explicit rules in place of few-shot correction, a validated box-detection approach, and, because it runs unattended for hours against an external API, the reliability fixes described in Section 9.

7.1 PDF input and page rendering

The source is a single 80-page PDF. Each page is rendered to a PNG at 300 DPI through PyMuPDF and cached to disk, so that a re-run does not re-render pages that already exist. Before committing to the full run, the notebook prints a time estimate. At roughly 6.5 minutes per page, 80 pages take about 8.7 hours unattended, which is the kind of run that has to be resumable. Section 9 covers the resume logic in detail.

7.2 Column and box detection

Each page is denoised and then split into its two text columns by locating the vertical gap between them, using a smoothed column-density profile in which the gap is found as the darkest region of the middle third of the page. Within each column, entries are separated by a hanging-indent rule. This directory prints every entry flush left, with any wrapped continuation line indented, so a line beginning at the left margin immediately after an indented or blank line marks the start of a new entry.

```
def detect_column_edges(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 0, 255,
                              cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
    core = binary[top_margin:bottom_margin, side_margin:w - side_margin]
    vproj = core.sum(axis=0)
    smoothed = np.convolve(vproj, np.ones(win) / win, mode="same")
    # the column gap sits in the darkest region of the middle third
    gap_idx = mid_start + int(np.argmin(smoothed[mid_start:mid_end]))
    ...

def detect_entries_in_column(col_img, smooth_window=7, min_ink_px=2):
    # per-row leftmost-ink position, smoothed to remove letter jitter
    # a row starts a new entry if it is at the left margin, and the row
    # above it was either blank or indented, never another margin row
    starts = [y for y in range(h) if at_margin[y]
              and (y == 0 or is_empty[y-1] or is_indented[y-1])]
```

This is the third revision of the function. It deliberately does not look for vertical whitespace between physical print lines, because this directory has essentially none. That approach was tried first and it collapsed entire columns into one or two very large boxes, as recorded in Section 9, item 1. The present revision also merges any two start detections landing implausibly close together, which fixes a case where the smoothed indent signal flickered for a row or two at a line transition and fired two starts a few pixels apart for what was in reality a single line break.

7.3 Per-crop OCR

Each detected box is cropped out of the page and sent to Gemini on its own, rather than transcribing a whole column at once and matching the resulting lines back to boxes by position afterwards. This guarantees that the text returned belongs to that exact crop. No separate matching step remains that could drift when the number of transcribed lines fails to match the number of boxes, which happens constantly, since entries span anywhere from one to three lines.

```
@retry(stop=stop_after_attempt(5),
       wait=wait_exponential(multiplier=2, min=5, max=60))
def ocr_crop(crop_img) -> List[str]:
    """Normally returns a list with ONE string. If a box ends up
    containing multiple merged entries, Gemini splits them and this
    returns multiple strings, so a bad box degrades to extra rows
    to review instead of lost data."""
    resp = client.models.generate_content(
        model=MODEL_NAME, contents=[CROP_OCR_PROMPT, pil],
        config=types.GenerateContentConfig(
            response_mime_type="application/json"),
    )
    texts = json.loads(resp.text)
    ...
```

The prompt asks for a JSON list of strings rather than a single string, specifically so that a box-detection error degrades gracefully. If a box accidentally contains more than one entry, every entry inside it is still kept, as recorded in Section 9, item 2. The retry decorator, provided by the tenacity library, wraps the whole call in exponential backoff so that a transient API error does not end an eight-hour run, as recorded in Section 9, item 7.

7.4 Field parsing

Each transcribed entry is parsed into structured fields by a second Gemini call, constrained to a Pydantic schema, DirectoryEntry, so that the response is always valid and typed JSON, and issued at temperature 0 so that identical input always parses identically. The prompt consists of explicit written rules rather than few-shot examples alone, for the reasons given in Section 6.

- Name order. This directory always prints the surname first, so "Addison Wiley" yields last_name = "Addison" and first_name = "Wiley", even where the given name resembles a surname in its own right. Position is the only signal used, not general knowledge about common names.
- Boarding, residence and rooms are three distinct fields. "bds 803 Main" yields boarding_raw, "rms over 1219 Hamilton" yields rooms_raw, and "r. 904 McKee" or "h. 1109 Fannin" yields residence_raw, with "h." additionally setting ownership_type to "home".
- Occupation and employer are split at the boundary, so "teamster Hipp & Key" yields occupation_raw = "teamster" and employer = "Hipp & Key".
- Racial markers preserve their parentheses exactly, giving "(c)" rather than "c".
- Ownership abbreviations expand, so "h." becomes "home" and "hhldr" becomes "householder".

A small number of correctly parsed few-shot examples drawn from the Step 2 ground truth are still supplied as supplementary context, selected to cover a diverse set of entry types including business, institution, cross-reference, and entries carrying notes, employer, workplace or ownership information. The rules above nevertheless do the actual correcting.

7.5 Cleanup: adjacent-duplicate removal

After every box has been transcribed and parsed, a final pass compares each entry against the one immediately preceding it in reading order. Where both carry the same last and first name, and their raw text overlaps substantially across the first 40 characters, they are treated as one person whose entry was split across two boxes, a truncated box followed by a fuller one. The shorter of the two is dropped and the more complete entry is retained, as recorded in Section 9, item 3.

7.6 Resumability

Every page's result is written to CSV as soon as that page finishes. On restart, the loop checks whether a page's CSV already exists on disk and, where it does, loads it instead of re-sending that page to Gemini, so that no API calls are wasted and nothing is re-billed. A page that legitimately produced zero real entries, every box having been rejected as not a directory entry because the page is an advertisement or a divider, has to be distinguished from a page that has simply not been processed yet. The resume logic originally got that distinction wrong, as recorded in Section 9, item 8.

7.7 Results

The full 80-page run produced 5,621 structured entries. 75 pages contained at least one real entry. Five pages, numbers 10, 11, 24, 64 and 65, were correctly recognised as containing none, being full-page advertisements or dividers rather than pipeline failures.

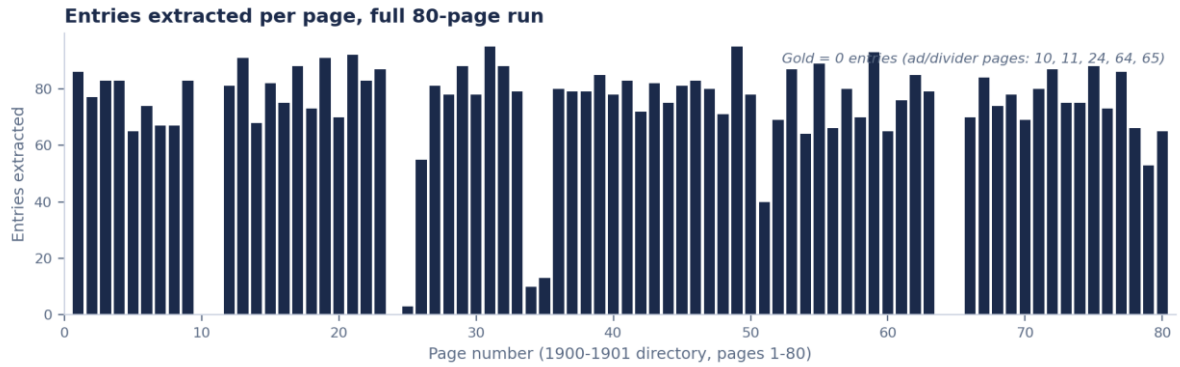

Figure 2. Entries extracted per page across the full 80-page run.

Output

For each page, a CSV and an Excel file carrying all 19 columns, being entry ID, source page, raw text, crop path and the 14 structured fields, together with an overlay PNG on which every detected box is drawn and numbered for visual quality assurance of box detection itself. Combined across every page processed in the run, the notebook also writes a single interactive review HTML file, presenting the source page image on the left and every extracted entry on the right, with crop thumbnail, raw transcription and parsed fields, so that results can be spot-checked against the original scan without opening 80 separate files.

8. Step 6: Dashboard and Excel Delivery

This notebook takes whatever Step 5 has produced on disk and converts it into the two things a reader will actually use, a browsable dashboard and a spreadsheet. It also computes everything that is not present in any single entry on its own, namely how entries relate to one another and which of them merit a second look.

8.1 Relationships

Relationships are computed strictly within a single directory edition. An 1892 listing and a 1900 listing are never treated as related even where the name is identical, because that is a different question.

- Co-residents, being entries that share a normalised address, whether residence, boarding or rooms.

- Coworkers and business links, being entries that share a normalised employer, with a fallback that extracts an affiliation from parenthetical text such as "Achenbach John (Achenbach & Schulte)" where no explicit employer field exists.
- Same-surname groups, being entries that share a normalised last name, which serve as a starting point for family research.

8.2 Data-quality flags: a heuristic that was wrong, and how it was found

Every entry is checked against two independent signals. The first, a fragment check, is close to deterministic. It asks whether the transcription begins mid-word or mid-phrase, or whether the entry is a person entry carrying only a surname and nothing else, which is the signature of a box drawn across a line break. That check worked as designed from the outset.

The duplicate check, which flags two neighbouring entries that appear to be the same listing captured twice, did not. It is worth setting out why, because the first version looked entirely reasonable and it was testing that caught it.

The first version required the names of two entries to match exactly before their text was compared, on the theory that a genuine duplicate, one listing read twice by overlapping boxes, would parse to the same name on both occasions. Tested against all 5,621 real entries, it flagged zero pairs. A real OCR duplicate, it turns out, more often disagrees on a letter within the name itself than agrees on it. Two reads of the same crop can return "Boule Emery" and "Boyle Emery". Requiring the names to match first was therefore filtering out precisely the case the check was meant to catch.

The fix drops the name requirement and compares full raw text similarity alone, at a threshold chosen empirically by examining every adjacent same-page pair across the real dataset rather than by guessing. This directory is alphabetised and genuinely dense with real, different, similarly worded neighbours, including siblings, a father and son sharing a name, and coworkers at a shared address, whose text can score as high as roughly 0.85 similar while describing two different real people. A threshold of 0.92 is approximately where that gives way to differences of only one or two characters, the signature of OCR noise acting on the same source text rather than of two distinct people. No threshold draws a perfectly clean line, and 0.92 trades recall for precision deliberately, so that the flag remains a credible prompt to check rather than noise.

The result is that 159 entries, or 2.8%, are flagged, comprising 113 on the fragment signal and 46 on the duplicate signal. This is down from 507, or 9.0%, under the first version of the duplicate check, which was mostly false positives.

8.3 The two products

Directory_Dashboard.html is a single self-contained file requiring no server, no internet connection and no external library, and opens in any browser. The Browse view provides search, sortable columns, checkbox multi-filters for entry type and racial marker, sounds-like search so that an OCR slip such as "Kbell" still finds "Abell", an abbreviations glossary with hover tooltips, and a full record detail view showing related entries. The Overview provides an always-visible summary chart grid alongside a fully interactive chart, in which the metric, the ordering, the number of bars and whether to split by marker can all be selected, with click-to-filter into Browse. Every filtered view can be exported to CSV, and every entry carries a permalink URL for citation.

Directory_Dashboard.xlsx holds the same 5,621 rows in spreadsheet form, with the fragment and duplicate flags as separate columns, relationship counts, and a Summary sheet of live COUNTIF and

SUMPRODUCT formulas that recalculate if a row is edited or added. Every field is written to be formula-injection safe. One residence was transcribed as literally "= 1000 Gray ave.", which Excel reads as a formula and fails on, as recorded in Section 9, item 11, so any field beginning with =, +, - or @ is forced to be written as text.

Both products are rebuilt from a DATASETS list declared at the top of the notebook, so that re-running it after Step 5 has processed further pages, or after a second directory edition has been added, is a one-line change rather than a rewrite.

9. Debugging Log: Twelve Real Bugs

These are recorded because they explain why the code described in Sections 7 and 8 takes the form it does, and because most of them are precisely the kind of failure worth watching for if this pipeline is extended to further pages or to a new edition.

1. Box splitting assumed the wrong page layout

An early revision attempted to find line breaks by grouping text rows wherever a vertical gap appeared between them, which assumes that real whitespace exists between physical print lines. This directory has no such whitespace. Its lines are tightly leaded, with no gap between one entry ending and the next beginning, so the approach fused entire columns into one or two very large boxes in place of the roughly 80 real entries on a page. A later revision of the fix below also had to merge a handful of false double-starts firing a few pixels apart at a single real line transition, visible as one entry, "Achenbach", being split into three fragments.

Fix. Reverted to per-row hanging-indent detection, described in Section 7.2, which reads indentation directly and does not depend on finding blank rows between lines. A minimum-gap merge was added for starts detected implausibly close together.

2. Merged boxes silently dropped entries

Where a box did contain more than one entry, through a box-detection error, the original per-crop OCR call returned a single transcribed string. The pipeline parsed that one string into one row and moved on, discarding every other person actually present in the crop without any error or warning. On one page, two boxes each swallowed roughly fifteen real entries in this way.

Fix. ocr_crop() now requests a JSON list of entries from Gemini, as described in Section 7.3, rather than a single string, and every entry in that list is parsed and kept. A bad box now costs extra rows to review rather than lost people.

3. The same entry counted twice

A wrapped entry, one continuing onto a second line such as a residence address, was sometimes split into two adjacent boxes: a truncated box holding only the first line, followed immediately by a fuller box holding the whole entry. Both were kept as separate rows, duplicating that person in the output.

Fix. Added the adjacent-duplicate cleanup pass described in Section 7.5. Where the last and first names match and the leading text overlaps, the two are merged and the more complete entry is kept.

4. Last and first names swapped, and parsing was not deterministic

The parsing prompt never stated this directory’s naming convention, that the surname is always printed first, so the model sometimes guessed on the basis of which word sounded like a first name.

"Addison Wiley" parsed with last_name = "Wiley", even though every other "Addison" entry on the same page parsed correctly. Separately, the same literal text, duplicated by a box-splitting error, parsed two different ways in two separate calls, because no temperature had been set on the parsing call.

Fix. Added an explicit name-order rule to the parsing prompt, given as the first rule in Section 7.4, and set temperature to 0, so that identical input always parses identically.

5. Few-shot correction made accuracy worse overall

The Step 4 experiment described in Section 6 added few-shot examples targeting five known error patterns. Mismatches fell, but macro-average field accuracy dropped from 86.6% to 77.5%, with some fields regressing by double digits.

Fix. Rebuilt the correction as explicit written rules within Step 5's base prompt rather than relying on few-shot examples to teach it, since rules do not overfit to a small example set in the way that a handful of examples can.

6. Encoding artefacts in the ground truth itself

The Step 2 HTML tool saved certain characters with broken UTF-8 encoding, producing corrupted sequences in place of an apostrophe and in place of the fraction one-half, which corrupted text-based comparisons in Steps 3 and 4 wherever those characters appeared.

Fix. Added a cleanup pass that repairs the known broken sequences in the ground truth JSON before every comparison.

7. A transient API failure could end an eight-hour run

The full 80-page run calls Gemini thousands of times across many hours. Early on, a single mid-run 503 from a transient API error ended the entire job, losing every page already processed.

Fix. Wrapped the OCR call in a retry decorator with exponential backoff, using five attempts, a multiplier of 2 and a wait of 5 to 60 seconds, so that a transient failure is retried rather than ending the run.

8. A legitimately empty page crashed the resume logic

A page can finish with zero real entries, every box having been correctly rejected as not a directory entry because the page is a full-page advertisement or a divider. Pages 10, 11, 24, 64 and 65 of the final run are such pages. The resume logic originally treated that outcome as an error rather than as a valid and already-finished result, and crashed while attempting to read an empty CSV back in.

Fix. Explicitly tolerate pd.errors.EmptyDataError on resume, and treat a page processed with zero rows as a normal, complete outcome rather than a failure.

9. The Excel export crashed after all 80 pages had finished

The per-page OCR and parsing loop completed successfully for all 80 pages and every result had already been saved to CSV. Only the final combined Excel export step, which runs after the loop, failed.

Fix. Added a recovery cell that rebuilds the full dataset from the CSVs already held on disk and re-runs the export alone. Nothing is re-sent to Gemini and nothing is re-billed.

10. Blank fields silently became the text "nan"

Pandas reads an empty CSV cell as a float NaN, and NaN is truthy in Python, so a naive str(value or "") produced the literal string "nan" in place of an empty string. Every blank field was then grouped under that fake shared value while relationships were computed in Step 6, so that every entry with a blank employer, for example, appeared to be a coworker of every other entry with a blank employer.

Fix. Added NaN-aware normalisation helpers, used everywhere a field is read, so that a genuinely blank field stays blank instead of becoming a false match key.

11. A residence broke Excel by being read as a formula

One entry's residence was transcribed as literally "= 1000 Gray ave." Excel reads a leading equals sign as the start of a formula, so that cell displayed an error in place of the actual address.

Fix. Any field value beginning with =, +, - or @ is now written with a leading apostrophe, which forces Excel to retain it as text.

12. The duplicate-detection flag caught zero true duplicates

This is covered in detail in Section 8.2. The first version of the near-duplicate check required the names of two entries to match before their text was compared. Tested against all 5,621 real entries it flagged zero pairs, because a genuine OCR duplicate more often disagrees on a letter within the name than agrees on it.

Fix. Dropped the name requirement and compared full raw text similarity alone, at a threshold of 0.92 chosen by measuring similarity across every real adjacent pair in the dataset rather than by assuming a number.

10. Final Results by the Numbers

Metric	Value
Pages processed	80
Pages with at least one real entry	75
Pages correctly recognised as empty (advertisements or dividers)	5 (pages 10, 11, 24, 64, 65)
Total structured entries	5,621
Person entries	5,308
Business entries	230
Institution entries	39
Entries with a printed racial marker	1,740
Entries flagged for review	159 (2.8%): 113 fragment, 46 duplicate
Entries with a co-resident found	1,786
Entries with a coworker found	1,766
Entries sharing a surname with another entry	4,498

11. Recommendations and Next Steps

1. Re-run the Step 3 and Step 4 accuracy comparison against the current, fully hardened Step 5 prompt. The figures in Sections 5 and 6 predate the explicit-rules rewrite and the determinism fix, recorded as items 4 and 5 in Section 9. A fresh pass would show how much those changes actually helped and would give a clean current baseline in place of a historical one.
2. Move to a deterministic entry ID before the dashboard's permalinks and citations are used anywhere permanent. IDs are currently row indexes, so re-running the pipeline shifts every ID. An ID derived from page and box, such as 1900-1901_p12_L_007, would survive a re-run.
3. Wire the saved entry crop image into the dashboard's flagged-entry view. Step 5 already saves entry_crop_path for each entry, and displaying that crop beside a flagged entry would make it directly verifiable against the scan without leaving the browser.
4. Extend the pipeline to another edition, whether the 1873 directory or a later Houston edition, in order to test how well it generalises. The DATASETS list in Step 6 is already built for this, so adding an edition is a one-line change. Relationships and flags must nevertheless remain computed within a single edition, as they already are, rather than merging across years.
5. Add a household or street-index view. This was requested during the summer and deferred for want of time. If the work is resumed it is the highest-value addition to the dashboard, since it mirrors the way the original directory was itself organised and builds on data the pipeline already produces, namely normalised address and street name.
6. Tighten box detection where possible. The process is now safety-netted, in that a merged or split box no longer loses or duplicates data, but every merge still costs an additional Gemini call and a deduplication check. Fewer merges mean lower API cost and less for a reviewer to check.

12. Appendix: Repository Structure

Six notebooks, run in order, each reading the previous step's output.

Notebook	Reads	Writes
step1_fix_boxes.ipynb	Source page image	output_box_fixing/corrected_boxes.json
step2_type_ground_truth_v3.ipynb	corrected_boxes.json	output_ground_truth/ground_truth.json
step3_accuracy_comparison.ipynb	corrected_boxes.json, ground_truth.json	output_step3_accuracy/ (mismatches, per-field accuracy)
step4_correction_loop.ipynb	ground_truth.json, Step 3 output	output_step4_correction/ (before and after accuracy)
step5_process_new_pages_14.ipynb	Source PDF, ground_truth.json for few-shot examples	step5_new_pages_output/ (csv, excel, overlays, entry_crops, review, page_stats.csv)
step6_build_dashboard.ipynb	step5_new_pages_output/csv/	dashboard_output/Directory_Dashboard.html and .xlsx

Each notebook is self-documenting, with markdown cells explaining what each section does and why, interleaved with the code. This report and the notebooks should therefore agree. Where they do not, the notebook is the source of truth.